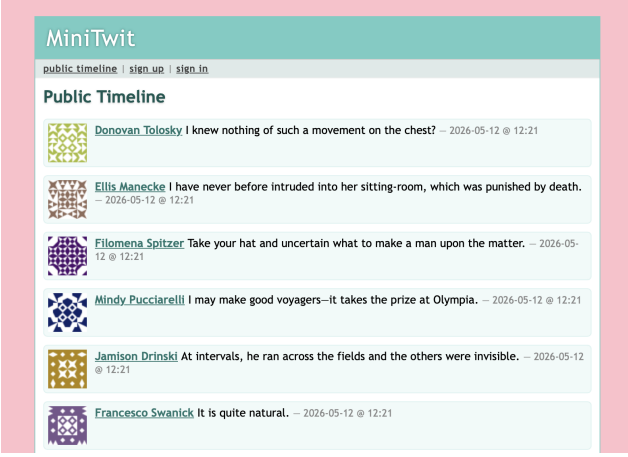
DevOps, Software Evolution and Software Maintenance, MSc



Group K - Pat Myaz

Carmen Alberte Nielsen | alcn@itu.dk
Casper Storm Frøding | csfr@itu.dk
Mads Christian Nørklit Jensen | macj@itu.dk
Mathilde Julie Gonzalez-Knudsen | mgon@itu.dk
Max Brix Koch | mbko@itu.dk

18-05-2026

Contents

1 System's Perspective **3**

1.1 Design and Architecture 3

1.2 Dependencies 3

1.3 Current State 3

2 Process' Perspective **3**

2.1 CI/CD 3

2.2 Monitor 3

2.2.1 Dashboards 6

2.3 Logging 8

2.4 Security 8

2.5 Availability 8

3 Reflection Perspective **8**

3.1 Evolution and Refactoring 8

3.2 Operation 9

3.3 Maintenance 9

4 Use of AI **9**

1 System's Perspective

A description and illustration of the:

- Design and architecture of your ITU-MiniTwit systems.
- All dependencies of your ITU-MiniTwit systems on all levels of abstraction and development stages. That is, list and briefly describe all technologies and tools you applied and depend on.
- Describe the current state of your systems, for example using results of static analysis and quality assessments.

1.1 Design and Architecture

1.2 Dependencies

1.3 Current State

2 Process' Perspective

This perspective should clarify how code or other artifacts come from idea into the running system and everything that happens on the way.

In particular, the following descriptions should be included:

- A complete description and illustration of stages and tools included in the CI/CD pipelines, including deployment and release of your systems.
- How do you monitor your systems and what precisely do you monitor?
- What do you log in your systems and how do you aggregate logs?
- Brief description of how you security hardened your systems.
- How do you handle availability and scaling in your systems?

2.1 CI/CD

2.2 Monitor

Under here is a sequence diagram of the process of collecting, forwarding, and visualizing application metrics in the monitoring system.

To ensure we do not leak information on our metrics of Minitwit, we have set a username and password on Grafana, that is automatically set on creation. In addition to this, only the Monitor's droplet IP adress is whitelisted to scrape https://monitor.bigtwit.app/, done via Traefik, so no other attacker would be able to access this information.

On this metrics endpoint we use `app.UseHttpMetrics();`, which writes things to the endpoint such as total amount of HTTP requests recieved, the duration of these requests, and more HTTP specific metrics. In addition to this, we added 2 "gauges" to the metrics, that showed the amount of tweets and users the app has. This data is cached, so if another metrics request is received in less than 30 seconds, we re-use the old data for tweets, and followers. This is done to not overwork the database, with requests to count the whole amount of tweets and users.

As can be seen from this code and the Sequence diagram, we have a hybrid of push/pull monitoring, as Minitwit pushes some internal data to a metrics endpoint, and Promtheus pulls from this endpoint.

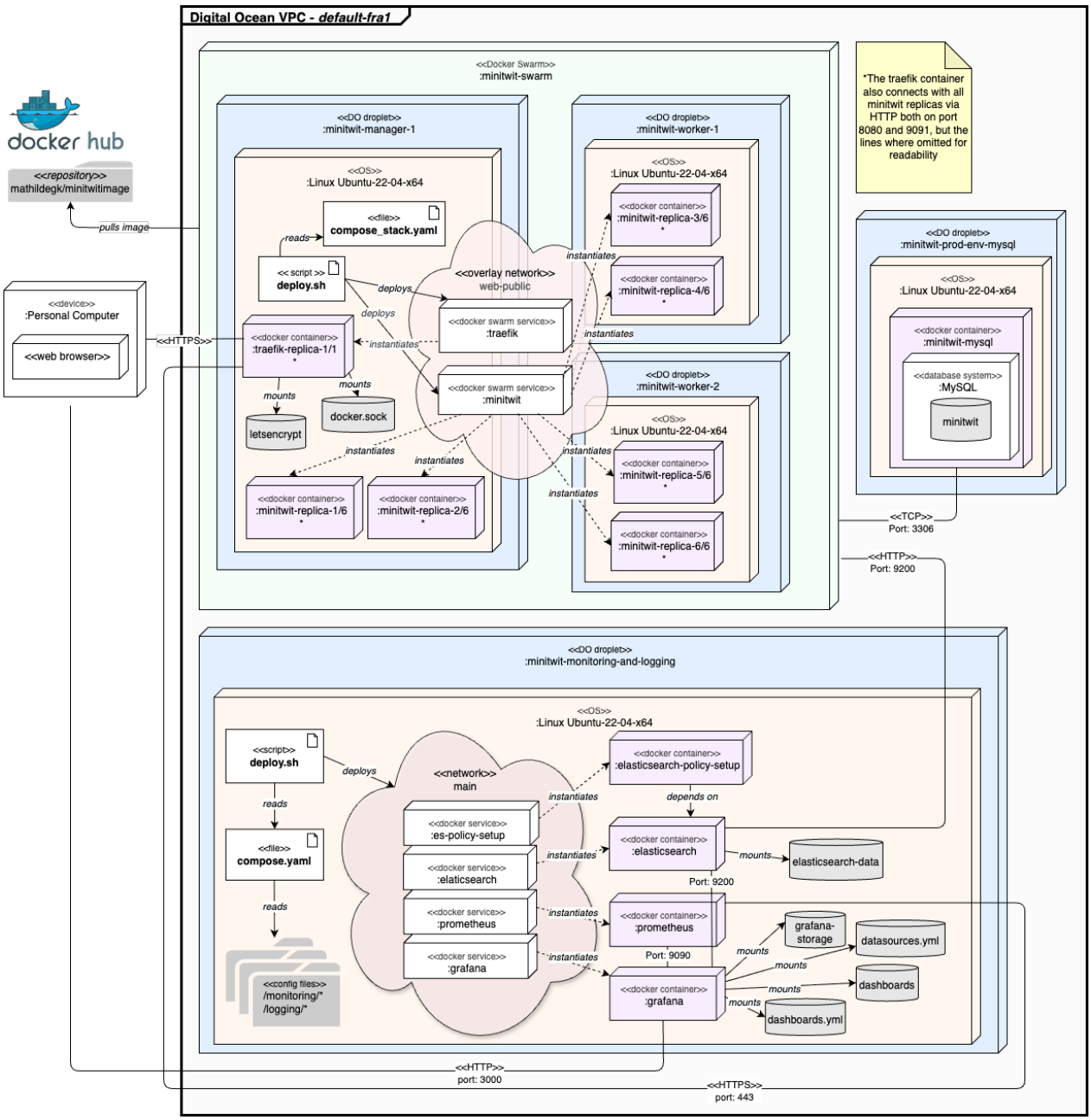


Figure 1: Allocation View

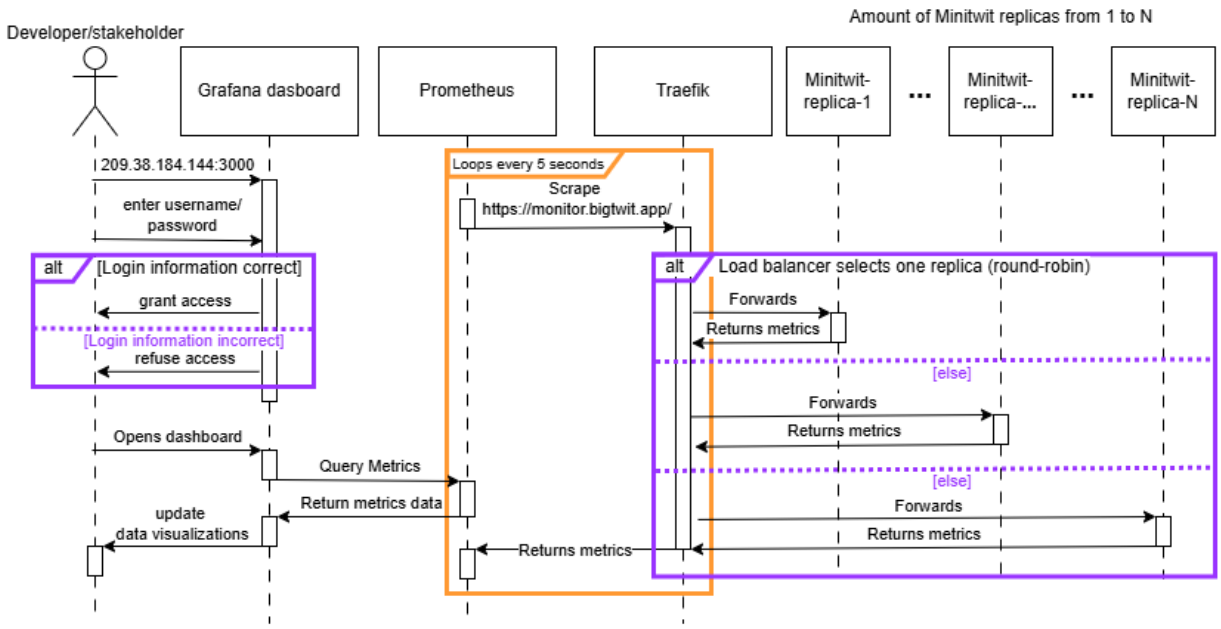


Figure 2: A sequence diagram of how monitoring works for Minitwit

```
// Before collecting metrics, we want to update the total amount of tweets and users,
// so we get the latest values when scraping
DateTime _lastUpdate = DateTime.MinValue;
Metrics.DefaultRegistry.AddBeforeCollectCallback(async (cancel) =>
{
  if ((DateTime.Now - _lastUpdate).TotalSeconds < 30) return; // Skip if too recent
  //The DbContext is a scoped service, so we need to create a scope to get it
  using (var scope = app.Services.CreateScope())
  {
    var context = scope.ServiceProvider.GetRequiredService<MinitwitContext>();

    // Query the database for the absolute latest numbers
    var totalTweets = await context.Messages.CountAsync(cancel);
    var totalUsers = await context.Users.CountAsync(cancel);

    // Update the Gauges
    MinitwitMetrics.TotalTweets.Set(totalTweets);
    MinitwitMetrics.TotalUsers.Set(totalUsers);
    _lastUpdate = DateTime.Now;
  }
});
```

Figure 3: The source code for caching, in the metrics

As can be seen in the diagram, Traefik load balances between each container, which creates this fluctuating graph, that can be seen in the dashboard under here, due to Prometheus not knowing exactly what container it has scraped, and can therefore not make an average.

To mitigate this error, we should have moved our Prometheus container from its' own droplet to the Manager droplet, and then from here it would be able to scrape all the individual replicas, and find averages over these, via the Docker network, and communicate this to Grafana on the monitoring droplet. However, this would still be a problem due to the workers being on different droplets than the manager, therefore we would need a package that could discover the exact location of each replica, for example the prometheus-docker-sd package (Link).

In addition to this we also wanted to monitor all of our services, i.e. not "just" the webservice, but also the database, to make it easier to determine where an error/bottleneck is located, however we did not get to implement this.

Due to us developers not wanting to constantly monitor our system, we have set up an *Uptime Check*, that monitors our droplets, and gives everyone in the team a notification if:

- https://bigtwit.app is down
- there is latency over 1000ms when accessing https://bigtwit.app
- The SSL certificate of bigtwit.app is about to expire.(However Traefik should automatically renew this certificate, before it expires.)

In addition to this we have added a ressource alert, that monitors our test database and production database, to see when it crosses over 70% CPU in over 5 minutes, which could indicate an attack, or a rapid increase in requests that it maybe cannot handle.

2.2.1 Dashboards

To see the dashboard, go to 46.101.69.11:3000, where the dashboard and logs can be viewed. The credentials are specified in the post made by Helge.

We made 2 different dashboards, one made with a developer focus (Infrastructure Health), and one made with a Business stakeholder focus(Product Performance).

2.2.1.1 Minitwit Infrastructure Health Dashboard

Below are some images of the dashboard and descriptions of the information it provides the developers:

Minitwit Infrastructure Health dashboard - Service health and HTTP Success Rate

Figure 4: Minitwit Infrastructure Health dashboard - Service health and HTTP Success Rate

Minitwit Infrastructure Health dashboard - Requests per second and Total HTTP requests received

Figure 5: Minitwit Infrastructure Health dashboard - Requests per second and Total HTTP requests received

Minitwit Infrastructure Health dashboard - Response time (P95 latency)

Figure 6: Minitwit Infrastructure Health dashboard - Response time (P95 latency)

- Immediate Root Cause Analysis: If the Status is UP but the Error Rate is spiking, developers know the app is running but the code or database is failing.

- Capacity Planning: Requests per second and Total Requests help developers understand if the current infrastructure (CPU/RAM) can handle the load or if they need to scale.
- Performance Regressions: By monitoring Latency(P95) and HTTP Success rate, developers can see if a recent code deployment made the app slower, even if it didn't *break* it.

We have, in the source code, created a */healthz* endpoint, that shows when Minitwit is up and running, however this is only used for spinning up the system locally with docker compose. Our dashboard's health check is currently based on whether or not Prometheus was able to scrape the target. If we had more time, we would have used the */healthz* endpoint instead, which would give a more relevant answer, which is seperated from Prometheus.

2.2.1.2 Minitwit Product Performance Dashboard

Below are some images of the dashboard and descriptions of the information it provides the business stakeholders:

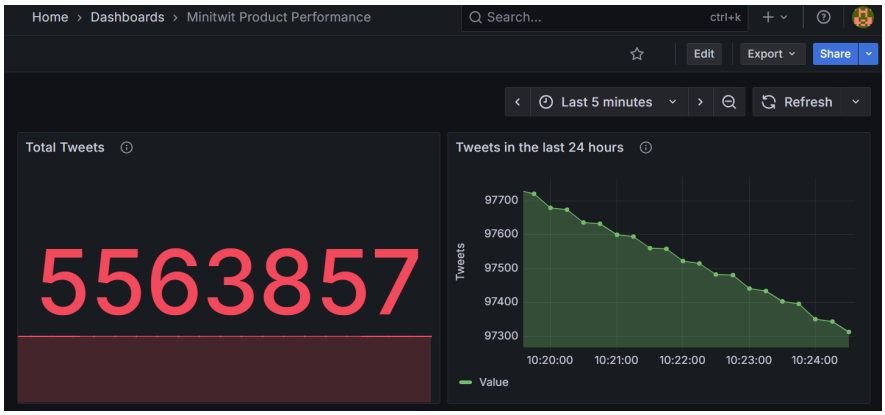


Figure 7: Minitwit Product Performance dashboard - Tweet overview

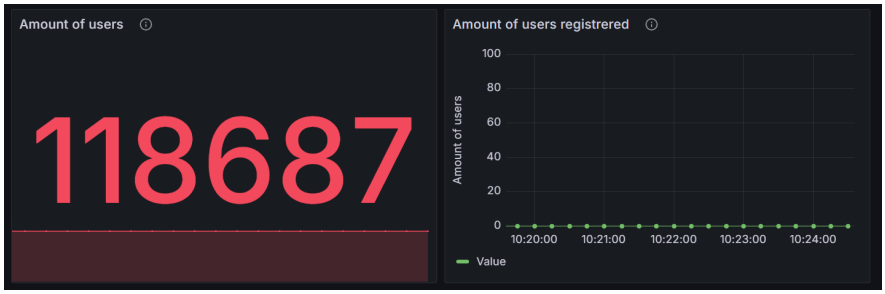


Figure 8: Minitwit Product Performance dashboard - User overview

- KPI Tracking: "Total Tweets" and "amount of users" are direct Key Performance Indicators. In a real application, if registrations drop to zero, the Marketing or Product team needs to know immediately.
- Engagement Trends: The 24-hour tweet increase shows the pulse of the community. A sudden drop might indicate a trend issue or a subtle bug that doesn't trigger an error but prevents posting. Also, amount of users registerered in the last hour, indicates if a campaign or other trend has worked or not, and if we therefore can see an increase or a drop.
- User Satisfaction: Slow apps lead to lost users, so we include an overview of the latency of HTTP requests to keep the business aligned with the need for technical optimization.

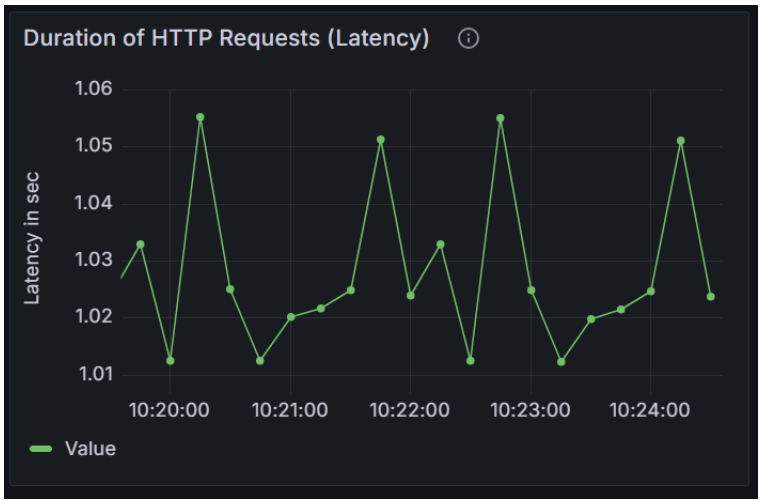


Figure 9: Minitwit Product Performance dashboard - Duration of HTTP Requests (Latency)

2.3 Logging

2.4 Security

2.5 Availability

3 Reflection Perspective

Describe the biggest issues, how you solved them, and which are major lessons learned with regards to:

- evolution and refactoring
- operation, and
- maintenance

of your ITU-MiniTwit systems. Link back to respective commit messages, issues, tickets, etc. to illustrate these.

Also reflect and describe what was the "DevOps" style of your work. For example, what did you do differently to previous development projects and how did it work?

3.1 Evolution and Refactoring

Remember to write about how changing from one docker container, to using Docker swarm destroyed our old way of monitoring. (OBS! we have written something about monitoring, so just a quick comment on how this destroyed it.)

3.2 Operation

3.3 Maintenance

4 Use of AI

ITU's rules on the use of generative AI apply for this report too. They are described here and in detail here. Please follow them. For your report that means that you have to state which generative AI tools have been used for which task(s) in your projects. Additionally, describe how generative AI tools have been used and briefly reflect and discuss how they supported or hindered your work process.